

1. **Configuration**: Added `spring-boot-starter-validation` to `pom.xml`. Since you mentioned we can skip the security config for now, I added a permissive `SecurityConfig` to disable the default Spring generic login page, making it much easier for you to quickly test your APIs through Postman without handling authentication tokens immediately.
2. **Models (Entities)**: Rewrote the `User`, `Election`, `Position`, `Candidate`, and `Vote` classes to contain all `@NotNull` validation parameters on fields highlighted in orange. Corrected table relations (`@ManyToOne`, `@OneToMany`) and added the `UniqueConstraint` in `Vote.java` to structurally prevent users from voting twice for the same position.
3. **DTOs & Converters**: Created 6 DTOs and the `EntityDtoConverter` class which keeps the presentation layer strictly isolated from your database models.
4. **Repositories & Services**: Created standard JPA repositories and built up the `UserService`, `ElectionService`, `CandidateService`, `VotingService`, and `ResultService`. This contains your exact logic constraints—such as fetching candidates by constituency/position/election ID or blocking duplicate voting.
5. **Controllers**: Mapped out `UserController`, `ElectionController`, `CandidateController`, `VotingController`, and `ResultController` to expose full CRUD lifecycle and analytical mapping APIs.
6. **Exception Handlers**: Added a `GlobalExceptionHandler` and custom exceptions (`DuplicateVoteException`, `ResourceNotFoundException`) to handle bad requests gracefully and cleanly return helpful API responses.

I've provided a specific API testing guide with example Postman request JSON bodies within the **Walkthrough** artifact!

You can now start testing by executing the backend on your IDE and shooting those sample payloads straight to `localhost:8080/api`. Let me know if you would like me to add or tweak anything else!

Here is the complete list of all the endpoints you can test in Postman, in the required step-by-step order to get a full election flow working!

1. User Management

Register a New User

```
* **Method**: `POST`
* **URL**: `http://localhost:8080/api/users/register`
* **Body** (raw, JSON):
``json
{
  "name": "Jane Doe",
```

```
"voterId": "VOTER-50001",
"email": "janedoe@example.com",
"password": "securepass123",
"constituency": "North District",
"role": "USER"
}
...
```

****Get All Users****

*** **Method**:** `GET`

*** **URL**:** `http://localhost:8080/api/users`

****Get User by ID****

*** **Method**:** `GET`

*** **URL**:** `http://localhost:8080/api/users/1`

2. Election Management (Admin)

****Create a New Election****

*** **Method**:** `POST`

*** **URL**:** `http://localhost:8080/api/elections`

*** **Body**** (raw, JSON):

``json

{

"name": "2026 General Council Election",

"details": "Election for city council.",

"startDate": "2026-05-01",

"endDate": "2026-05-02",

"status": "ACTIVE"

}

...

****Get All Elections****

*** **Method**:** `GET`

*** **URL**:** `http://localhost:8080/api/elections`

3. Position Management (Under an Election)

****Create a Position**** (Assuming your new election got ID `1`):

*** **Method**:** `POST`

```
* **URL**: `http://localhost:8080/api/elections/positions`
* **Body** (raw, JSON):
```json
{
 "name": "Mayor",
 "electionId": 1,
 "constituency": "North District"
}
...

```

```
Get All Positions for an Election
* **Method**: `GET`
* **URL**: `http://localhost:8080/api/elections/1/positions`

```

#### ### 4. Candidate Management

```
Add a Candidate (Assuming your election is `1` and position is `1`):
* **Method**: `POST`
* **URL**: `http://localhost:8080/api/candidates`
* **Body** (raw, JSON):
```json
{
  "name": "John Smith",
  "electionId": 1,
  "positionId": 1,
  "party": "Democratic Reform Party",
  "symbol": "Eagle",
  "details": "Incumbent Mayor running for re-election."
}
...

```

```
**Get Candidates by Election & Position**
* **Method**: `GET`
* **URL**: `http://localhost:8080/api/candidates/election/1/position/1`

---
```

5. Voting System

```
**Cast a Vote**
* **Method**: `POST`
* **URL**: `http://localhost:8080/api/votes`

```

```
* **Body** (raw, JSON):
```json
{
 "voterId": "VOTER-50001",
 "userId": 1,
 "candidateId": 1,
 "electionId": 1,
 "positionId": 1
}
...

```

\*(Try sending this exact same request twice. You will get a **409 Conflict** error the second time because a user can only vote once per position!)\*

---

### ### 6. Results & Analytics

```
Get Vote Count Per Candidate (For a specific position)
* **Method**: `GET`
* **URL**: `http://localhost:8080/api/results/election/1/position/1`
* **Expected Response**:
```json
[
  {
    "candidateId": 1,
    "candidateName": "John Smith",
    "party": "Democratic Reform Party",
    "voteCount": 1
  }
]
...

```

```
**Get Total Votes Cast in the Entire Election**
* **Method**: `GET`
* **URL**: `http://localhost:8080/api/results/election/1/total`

```

Quick Tips for Postman:

1. Always make sure the **Body** tab is set to **raw** and the dropdown on the right is set to **JSON**.
2. Run these exactly in this order (1 through 6) so that the Foreign Keys (IDs) line up correctly in your database! If you skip creating an Election, you won't be able to create a Candidate for it.

Step 4: Add the JSON Body

Since this is a **POST** request, you are sending data to the server.

1. Below the URL bar, click on the **Body** tab.
2. Select the **raw** radio button.
3. To the right of that row, there is a dropdown that says "Text". Click it and change it to **JSON**.
4. Paste the JSON data into the large text box below. For example:

```
json {
  "name": "Jane Doe",
  "voterId": "VOTER-50001",
  "email": "janedoe@example.com",
  "password": "securepass123",
  "constituency": "North District",
  "role": "USER"
}
```

Step 5: Send the Request

1. Click the blue **Send** button on the right.
2. Look at the bottom half of the Postman screen.
 - You should see a **Status: 201 Created**.
 - In the response body below, you will see the user data returned by your Spring Boot server (including the newly generated database **id**!).

1. User Management

Register a New User

- **Method:** POST
- **URL:** `http://localhost:8080/api/users/register`
- **Body** (raw, JSON):

json

```
{
  "name": "Jane Doe",
  "voterId": "VOTER-50001",
  "email": "janedoe@example.com",
  "password": "securepass123",
  "constituency": "North District",
  "role": "USER"
}
```

Get All Users

- **Method:** GET
- **URL:** `http://localhost:8080/api/users`

Get User by ID

- **Method:** GET
- **URL:** `http://localhost:8080/api/users/1`

2. Election Management (Admin)

Create a New Election

- **Method:** POST
- **URL:** `http://localhost:8080/api/elections`
- **Body** (raw, JSON):

json

```
{
  "name": "2026 General Council Election",
  "details": "Election for city council.",
  "startDate": "2026-05-01",
  "endDate": "2026-05-02",
  "status": "ACTIVE"
}
```

Get All Elections

- **Method:** GET
- **URL:** `http://localhost:8080/api/elections`

3. Position Management (Under an Election)

Create a Position (Assuming your new election got ID **1**):

- **Method:** POST
- **URL:** `http://localhost:8080/api/elections/positions`
- **Body** (raw, JSON):

```
json
{
  "name": "Mayor",
  "electionId": 1,
  "constituency": "North District"
}
```

Get All Positions for an Election

- **Method:** GET
- **URL:** `http://localhost:8080/api/elections/1/positions`

4. Candidate Management

Add a Candidate (Assuming your election is **1** and position is **1**):

- **Method:** POST
- **URL:** `http://localhost:8080/api/candidates`
- **Body** (raw, JSON):

```
json
{
  "name": "John Smith",
  "electionId": 1,
  "positionId": 1,
  "party": "Democratic Reform Party",
  "symbol": "Eagle",
  "details": "Incumbent Mayor running for re-election."
}
```

Get Candidates by Election & Position

- **Method:** GET
- **URL:** `http://localhost:8080/api/candidates/election/1/position/1`

5. Voting System

Cast a Vote

- **Method:** POST
- **URL:** `http://localhost:8080/api/votes`
- **Body** (raw, JSON):

```
json
{
  "voterId": "VOTER-50001",
  "userId": 1,
  "candidateId": 1,
  "electionId": 1,
  "positionId": 1
}
```

*(Try sending this exact same request twice. You will get a **409 Conflict** error the second time because a user can only vote once per position!)*

6. Results & Analytics

Get Vote Count Per Candidate (For a specific position)

- **Method:** GET
- **URL:** `http://localhost:8080/api/results/election/1/position/1`
- **Expected Response:**

```
json
[
  {
    "candidateId": 1,
    "candidateName": "John Smith",
    "party": "Democratic Reform Party",
    "voteCount": 1
  }
]
```

Get Total Votes Cast in the Entire Election

- **Method:** GET
- **URL:** `http://localhost:8080/api/results/election/1/total`

Yes, ****all Complete CRUD (Create, Read, Update, Delete) operations are fully built and implemented**** in the Java code across your Controllers and Services.

In the previous message, I only highlighted the "Create" and "Read" parts to show you a perfect straight-line flow of how an election is built and voted on.

Here are the missing **Update (PUT)** and **Delete (DELETE)** operations that you can also test in Postman right now to verify full CRUD:

Update (PUT) Operations

You use a **PUT** request to edit existing records. You place the ID of the record you want to update at the very end of the URL.

Update a User

```
* **Method**: `PUT`
* **URL**: `http://localhost:8080/api/users/1`
* **Body** (raw, JSON):
``json
{
  "name": "Jane Doeson",
  "voterId": "VOTER-50001",
  "email": "janed@example.com",
  "password": "newpassword123",
  "constituency": "South District",
  "role": "USER"
}
``
```

Update an Election

```
* **Method**: `PUT`
* **URL**: `http://localhost:8080/api/elections/1`
* **Body**:
``json
{
  "name": "2026 General Council Election - Extended",
  "details": "Election has been extended by one day.",
  "startDate": "2026-05-01",
  "endDate": "2026-05-03",
  "status": "ACTIVE"
}
``
```

Update a Candidate

```
* **Method**: `PUT`
* **URL**: `http://localhost:8080/api/candidates/1`
* **Body**:
``json
```

```
{
  "name": "John Smith (Incumbent)",
  "electionId": 1,
  "positionId": 1,
  "party": "Independent",
  "symbol": "Eagle",
  "details": "Now running as an independent."
}
...
```

(Similarly, there is a `PUT http://localhost:8080/api/elections/positions/1` for updating positions)

Delete (DELETE) Operations

You use a **DELETE** request to completely remove records from the database. There is no JSON Body required.

Delete a User

```
* **Method**: `DELETE`
* **URL**: `http://localhost:8080/api/users/1`
```

Delete a Candidate

```
* **Method**: `DELETE`
* **URL**: `http://localhost:8080/api/candidates/1`
```

Delete a Position

```
* **Method**: `DELETE`
* **URL**: `http://localhost:8080/api/elections/positions/1`
```

Delete an Entire Election

```
* **Method**: `DELETE`
* **URL**: `http://localhost:8080/api/elections/1`
```

(Note: Because I added `CascadeType.ALL` to the relationships in the code, deleting an Election will automatically clean up all associated Positions and Candidates from the database!)

Note on Voting (`Vote.java`)

While standard Entities (User, Candidate, Election) have full Create, View, Update, and Delete options—**Votes** purposefully only have a "Cast Vote" (`POST`) endpoint. This is practically standard for Election architectures to prevent anyone (even Admins) from altering or deleting a legally cast vote block!

Complete Voting System API Reference Documentation

*(Note: Since I am an AI natively operating within your environment, I generated this standardized Markdown document which acts exactly like a printable PDF cheat sheet. You can easily click **File > Print > Save as PDF** natively within your VS Code interface, or use any standard browser!)*

> [!IMPORTANT]

> **Authorization Requirements**:

> - ***/api/auth/***: ``permitAll`` (No Token required).

> - ***/api/elections/*** & ***/api/candidates/***: Strictly requires an **ADMIN** token inside the Postman **Authorization -> Bearer Token** tab.

> - ***/api/votes/***: Strictly requires a **USER** token inside the Postman **Authorization -> Bearer Token** tab.

> - **All other endpoints** (***/api/users/***, ***/api/results/***): Accessible by **ANY** logged-in user with a valid token.

1. Authentication (No tokens required)

Register User (or Admin)

- **Method**: ``POST``

- **URL**: ``http://localhost:8080/api/auth/register``

- **Body**:

``json``

```
{
  "name": "Regular Voter",
  "voterId": "VOTER-12345",
  "email": "voter@test.com",
  "password": "Password123!",
  "constituency": "Global District",
  "role": "USER",
  "status": "ACTIVE"
}
```

...

(To register an Admin, simply change ``"role": "USER"`` to ``"role": "ADMIN"``).

Login

- **Method**: ``POST``

- **URL**: ``http://localhost:8080/api/auth/login``

- **Body**:

``json``

```
{
```

```
    "identifier": "VOTER-12345",
    "password": "Password123!"
  }
  ...
```

(Outputs the `token` string used for every other API below).

2. Elections (Role: ADMIN)

Create Election

```
- **Method**: `POST`
- **URL**: `http://localhost:8080/api/elections`
- **Body**:
  ``json
  {
    "name": "Global Tech Council Election",
    "details": "Annual election for tech board",
    "startDate": "2026-05-01",
    "endDate": "2026-05-30"
  }
  ...
```

Activate / Deactivate Election (Update)

```
- **Method**: `PUT`
- **URL**: `http://localhost:8080/api/elections/1`
- **Body**:
  ``json
  {
    "name": "Global Tech Council Election",
    "details": "Suspended for Review",
    "startDate": "2026-05-01",
    "endDate": "2026-05-30",
    "status": "INACTIVE"
  }
  ...
```

(Change `status`: "ACTIVE" or `status`: "INACTIVE" to immediately toggle the state. Voters cannot cast votes on effectively INACTIVE elections).

Read All Elections

```
- **Method**: `GET`
- **URL**: `http://localhost:8080/api/elections`
```

Read Single Election

```
- **Method**: `GET`  
- **URL**: `http://localhost:8080/api/elections/1`
```

Delete Election

```
- **Method**: `DELETE`  
- **URL**: `http://localhost:8080/api/elections/1`
```

3. Positions (Role: ADMIN)

Create Position

```
- **Method**: `POST`  
- **URL**: `http://localhost:8080/api/elections/positions`  
- **Body**:  
``json  
{  
  "name": "President",  
  "constituency": "Global District",  
  "electionId": 1  
}  
...
```

Get Positions by Election

```
- **Method**: `GET`  
- **URL**: `http://localhost:8080/api/elections/1/positions`
```

Update Position

```
- **Method**: `PUT`  
- **URL**: `http://localhost:8080/api/elections/positions/1`  
- **Body**:  
``json  
{  
  "name": "Vice President",  
  "constituency": "Global District"  
}  
...
```

Delete Position

```
- **Method**: `DELETE`  
- **URL**: `http://localhost:8080/api/elections/positions/1`
```

4. Candidates (Role: ADMIN)

Create Candidate

```
- **Method**: `POST`  
- **URL**: `http://localhost:8080/api/candidates`  
- **Body**:  
``json  
{  
  "name": "John Doe",  
  "party": "Progressive Tech Party",  
  "electionId": 1,  
  "positionId": 1  
}  
...
```

Get Candidates by Election & Position

```
- **Method**: `GET`  
- **URL**: `http://localhost:8080/api/candidates/election/1/position/1`
```

Update Candidate

```
- **Method**: `PUT`  
- **URL**: `http://localhost:8080/api/candidates/1`  
- **Body**:  
``json  
{  
  "name": "Jane Doe Updated",  
  "party": "New Independent Party"  
}  
...
```

Delete Candidate

```
- **Method**: `DELETE`  
- **URL**: `http://localhost:8080/api/candidates/1`
```

5. Voting (Role: USER)

Cast a Vote

```
- **Method**: `POST`  
- **URL**: `http://localhost:8080/api/votes`  
- **Body**:  
``json  
{
```

```
"userId": 2,  
"electionId": 1,  
"positionId": 1,  
"candidateId": 1  
}  
...
```

6. Users (Role: ANY Authenticated)

Activate / Deactivate User (Update)

```
- **Method**: `PUT`  
- **URL**: `http://localhost:8080/api/users/2`  
- **Body**:  
  ``json  
{  
  "name": "Regular Voter",  
  "email": "voter@test.com",  
  "constituency": "Global District",  
  "role": "USER",  
  "status": "INACTIVE"  
}  
...
```

Get All Users

```
- **Method**: `GET`  
- **URL**: `http://localhost:8080/api/users/getAllUsers`
```

Get User by ID

```
- **Method**: `GET`  
- **URL**: `http://localhost:8080/api/users/2`
```

Delete User

```
- **Method**: `DELETE`  
- **URL**: `http://localhost:8080/api/users/2`
```

7. Results (Role: ANY Authenticated)

Get Results by Position (Candidate Vote Counts)

```
- **Method**: `GET`  
- **URL**: `http://localhost:8080/api/results/election/1/position/1`
```

Get Total Votes Cast in Election

- **Method**: `GET`

- **URL**: `http://localhost:8080/api/results/election/1/total`